

---

**Master : SDIA**

**Module : Systèmes Multi-Agents et Intelligence Artificielle Distribuée**

---

*PROJET DE FIN DE MODULE :*

# Assistant Intelligent de Recrutement RH

*Réalisé par :*

Houdaifa BAHOU  
Anass MAKHLOUK

*Encadré par :*

Mme RETAL Sara

Année universitaire 2025 - 2026

# Table des matières

---

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction et Choix du Sujet</b>                       | <b>3</b>  |
| 1.1      | Contexte du projet . . . . .                                | 3         |
| 1.2      | Justification du choix du sujet . . . . .                   | 3         |
| 1.2.1    | Pertinence du domaine RH . . . . .                          | 3         |
| 1.2.2    | Adéquation avec les critères du projet . . . . .            | 3         |
| 1.3      | Vision globale du système . . . . .                         | 4         |
| <b>2</b> | <b>Architecture du Système Multi-Agent</b>                  | <b>5</b>  |
| 2.1      | Choix de l'orchestration : LangGraph . . . . .              | 5         |
| 2.1.1    | Pourquoi LangGraph ? . . . . .                              | 5         |
| 2.1.2    | Modèle d'orchestration : Hiérarchique . . . . .             | 5         |
| 2.1.3    | Structure du graphe LangGraph . . . . .                     | 5         |
| 2.2      | Présentation des 5 agents . . . . .                         | 8         |
| <b>3</b> | <b>RAG Agentique</b>                                        | <b>11</b> |
| 3.1      | Principe du RAG dans notre système . . . . .                | 11        |
| 3.2      | Architecture de la Base de Connaissances . . . . .          | 11        |
| 3.3      | Paramètres de Chunking et Indexation . . . . .              | 11        |
| 3.3.1    | Stratégie Actuelle . . . . .                                | 11        |
| 3.3.2    | Paramètres d'Indexation . . . . .                           | 12        |
| <b>4</b> | <b>Human-in-the-Loop (HITL)</b>                             | <b>13</b> |
| 4.1      | Philosophie du contrôle humain . . . . .                    | 13        |
| 4.2      | Checkpoint 1 — Validation de la liste courte (RH) . . . . . | 13        |
| 4.3      | Checkpoint 2 — Décision finale (Manager) . . . . .          | 13        |
| <b>5</b> | <b>Évaluation des Prompts</b>                               | <b>14</b> |
| <b>6</b> | <b>Interface Web — FastAPI + React</b>                      | <b>17</b> |
| 6.1      | Architecture technique . . . . .                            | 17        |
| 6.2      | API Backend FastAPI . . . . .                               | 17        |
| 6.2.1    | Architecture Générale . . . . .                             | 17        |
| 6.2.2    | Endpoints REST . . . . .                                    | 18        |
| 6.2.3    | WebSocket — Streaming Temps Réel . . . . .                  | 18        |
| 6.2.4    | Code — WebSocket Manager . . . . .                          | 19        |
| 6.2.5    | Code — Endpoint Upload CVs . . . . .                        | 20        |
| 6.2.6    | Code — Validation HITL 1 (RH) . . . . .                     | 20        |
| <b>7</b> | <b>Stack Technique et Architecture Complète</b>             | <b>22</b> |
| 7.1      | Stack technique détaillée . . . . .                         | 22        |
| 7.2      | Structure complète du projet . . . . .                      | 22        |
| 7.3      | Guide d'installation et d'exécution . . . . .               | 23        |

|          |                                       |           |
|----------|---------------------------------------|-----------|
| 7.3.1    | Installation backend . . . . .        | 23        |
| 7.3.2    | Installation frontend . . . . .       | 24        |
| <b>8</b> | <b>Démonstration</b>                  | <b>25</b> |
| <b>9</b> | <b>Conclusion et Perspectives</b>     | <b>31</b> |
| 9.1      | Apports pédagogiques . . . . .        | 31        |
| 9.2      | Perspectives d'amélioration . . . . . | 31        |
| 9.3      | Conclusion . . . . .                  | 31        |

# 1. Introduction et Choix du Sujet

---

## 1.1 Contexte du projet

Dans le cadre du module **Systèmes Multi-Agents et Intelligence Artificielle Distribuée** du Master SDIA, il nous a été demandé de concevoir et d'implémenter un **système multi-agent intelligent** capable de raisonner, de planifier et d'exécuter des tâches complexes de manière autonome. Ce projet représente l'aboutissement pratique des connaissances théoriques acquises sur les architectures agentiques modernes.

### Objectif pédagogique

Concevoir un écosystème multi-agent complet démontrant : l'orchestration via LangGraph, le RAG agentique, la supervision humaine (HITL), l'évaluation des prompts, et une interface web fonctionnelle.

## 1.2 Justification du choix du sujet

Parmi plusieurs domaines possibles (finance, santé, éducation, logistique), nous avons retenu le secteur des **Ressources Humaines** et plus précisément le processus de recrutement. Ce choix s'appuie sur plusieurs arguments solides.

### 1.2.1 Pertinence du domaine RH

Le recrutement est un processus naturellement **multi-étapes et multi-acteurs** qui se prête parfaitement à une décomposition en agents spécialisés. Chaque étape (analyse du poste, screening, entretien, décision) correspond à une expertise distincte qu'un agent peut modéliser.

#### Pourquoi le recrutement ?

- **Volume de données** : Les entreprises reçoivent des centaines de CVs par offre, rendant l'automatisation indispensable
- **Complexité décisionnelle** : La sélection nécessite d'analyser des dimensions multiples (compétences, soft skills, fit culturel)
- **Enjeux humains** : Les décisions impactent des vies, justifiant la présence d'un humain dans la boucle (HITL)
- **Richesse documentaire** : CVs, fiches de poste, grilles d'évaluation → base idéale pour le RAG

### 1.2.2 Adéquation avec les critères du projet

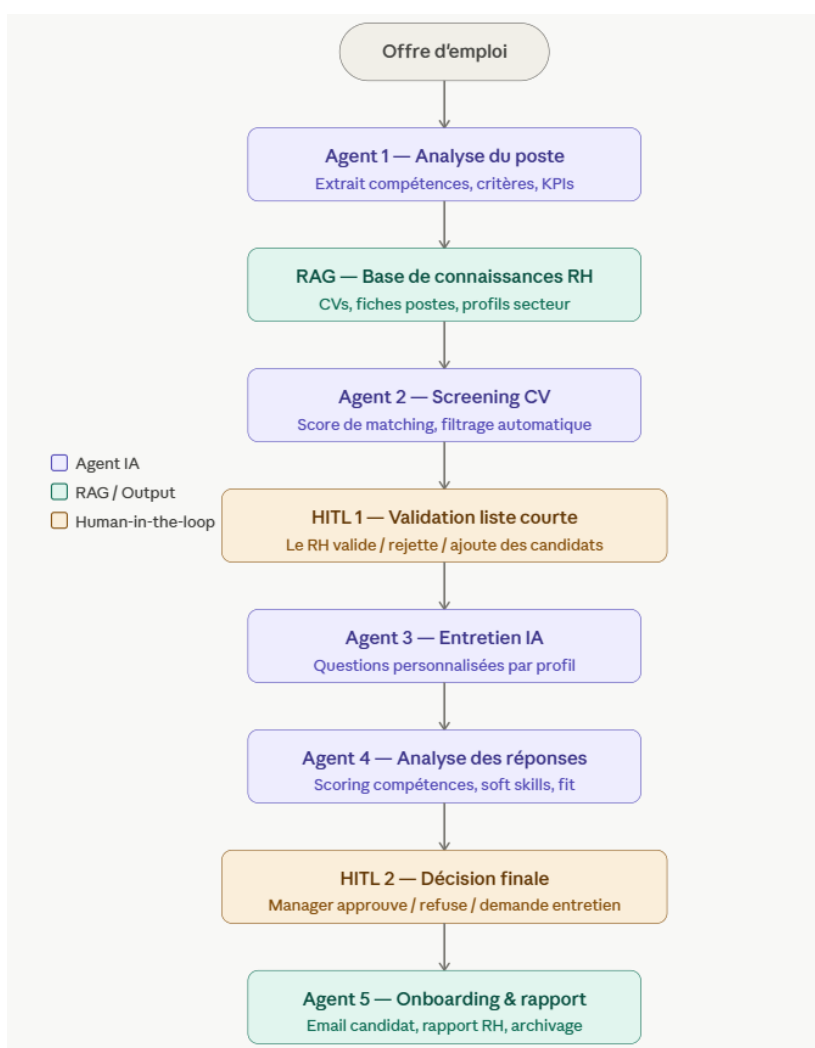
Le tableau suivant illustre comment le sujet couvre tous les critères requis :

TABLE 1.1 – Mapping critères du projet / fonctionnalités implémentées

| Critère requis                | Implémentation dans notre projet                                       |
|-------------------------------|------------------------------------------------------------------------|
| <b>Workflow agentique</b>     | 5 agents spécialisés orchestrés par LangGraph StateGraph               |
| <b>RAG agentique</b>          | ChromaDB avec 3 collections : CVs, fiches postes, grilles entretien    |
| <b>Human-in-the-loop</b>      | 2 points de contrôle : validation RH (liste courte) + décision manager |
| <b>Évaluation des prompts</b> | A/B testing via LangSmith + métriques RAGAS                            |
| <b>Interface web</b>          | FastAPI (backend) + React/Vite (frontend) avec Web-Socket temps réel   |

## 1.3 Vision globale du système

Notre **Assistant Intelligent de Recrutement RH** automatise l'intégralité du cycle de recrutement, depuis l'analyse de l'offre d'emploi jusqu'à la génération du rapport final, tout en maintenant un contrôle humain sur les décisions critiques.



## 2. Architecture du Système Multi-Agent

---

### 2.1 Choix de l'orchestration : LangGraph

#### 2.1.1 Pourquoi LangGraph ?

LangGraph est un framework d'orchestration d'agents basé sur la théorie des graphes d'état. Contrairement à une approche séquentielle simple (chaînes LangChain), LangGraph permet :

- Des **cycles et boucles** dans le flux d'exécution (ex : relancer les entretiens)
- La **persistance d'état** entre les étapes via un **StateGraph**
- Des **interruptions contrôlées** (`interrupt_before`) pour le HITL
- Un **roulage conditionnel** basé sur l'état courant

#### 2.1.2 Modèle d'orchestration : Hiérarchique

Nous avons opté pour une orchestration **hiérarchique** plutôt que séquentielle pure. Un **Supervisor Agent** implicite (via les routeurs conditionnels de LangGraph) décide de la transition entre les nœuds.

##### Choix architectural justifié

L'orchestration hiérarchique est préférable car le flux de recrutement comporte des **décisions conditionnelles** (aucun candidat retenu → fin prématurée, manager insatisfait → relance des entretiens). Un graphe conditionnel modélise mieux cette réalité qu'une chaîne strictement linéaire.

#### 2.1.3 Structure du graphe LangGraph

Listing 2.1 – Construction du StateGraph LangGraph

```
1 from __future__ import annotations
2
3 import logging
4
5 from langgraph.checkpoint.memory import MemorySaver
6 from langgraph.graph import END, START, StateGraph
7
8 from app.agents.cv_screener import cv_screener_node
9 from app.agents.interview_analyzer import interview_analyzer_node
10 from app.agents.interview_generator import interview_generator_node
11 from app.agents.job_analyzer import job_analyzer_node
12 from app.agents.report_generator import report_generator_node
13 from app.graph.hitl_manager import hitl_hr_node, hitl_manager_node
14 from app.graph.state import PipelineStep, RecruitmentState
```

```

15
16 logger = logging.getLogger(__name__)
17
18
19 NODE_JOB_ANALYZER = "job_analyzer"
20 NODE_CV_SCREENERS = "cv_screener"
21 NODE_HITL_HR = "hitl_hr_validation"
22 NODE_INTERVIEW_GENERATOR = "interview_generator"
23 NODE_INTERVIEW_ANALYZER = "interview_analyzer"
24 NODE_HITL_MANAGER = "hitl_manager_validation"
25 NODE_REPORT_GENERATOR = "report_generator"
26
27
28 def route_on_error(state: RecruitmentState) -> str:
29     if state.get("has_critical_error"):
30         logger.error(
31             "[Workflow] Erreur critique detectee " "interruption du
32                 pipeline."
33         )
34         return "end"
35     return "continue"
36
37 def route_after_hitl_hr(state: RecruitmentState) -> str:
38     hr_validation = state.get("hr_validation")
39     if hr_validation:
40         from app.models.hitl import HITLStatus
41
42         if hr_validation.status in (
43             HITLStatus.REANALYSIS_REQUESTED,
44             HITLStatus.REJECTED,
45         ):
46             logger.info("[Workflow] RH demande une reanalyse des
47                 CVs.")
48             return "reanalyze"
49     return "continue"
50
51 def route_after_hitl_manager(state: RecruitmentState) -> str:
52     manager_validation = state.get("manager_validation")
53     if manager_validation:
54         from app.models.hitl import ManagerDecision
55         from app.models.hitl import HITLStatus
56
57         if manager_validation.status == HITLStatus.REJECTED:
58             logger.info("[Workflow] Manager rejette le processus.")
59             return "cancel"
60
61         decision = manager_validation.decision
62         if decision == ManagerDecision.ADDITIONAL_INTERVIEW:
63             logger.info("[Workflow] Manager demande un entretien
64                 supplementaire.")
65             return "additional_interview"
66         if decision == ManagerDecision.CANCEL:
67             logger.info("[Workflow] Manager annule le processus.")
68             return "cancel"
69     return "hire"

```

```

70
71 def build_recruitment_graph() -> StateGraph:
72     graph = StateGraph(RecruitmentState)
73
74     graph.add_node(NODE_JOB_ANALYZER, job_analyzer_node)
75     graph.add_node(NODE_CV_SCREENERS, cv_screener_node)
76     graph.add_node(NODE_HITL_HR, hitl_hr_node)
77     graph.add_node(NODE_INTERVIEW_GENERATOR,
78                     interview_generator_node)
79     graph.add_node(NODE_INTERVIEW_ANALYZER, interview_analyzer_node)
80     graph.add_node(NODE_HITL_MANAGER, hitl_manager_node)
81     graph.add_node(NODE_REPORT_GENERATOR, report_generator_node)
82
83     graph.add_edge(START, NODE_JOB_ANALYZER)
84
85     graph.add_conditional_edges(
86         NODE_JOB_ANALYZER,
87         route_on_error,
88         {
89             "continue": NODE_CV_SCREENERS,
90             "end": END,
91         },
92     )
93
94     graph.add_conditional_edges(
95         NODE_CV_SCREENERS,
96         route_on_error,
97         {
98             "continue": NODE_HITL_HR,
99             "end": END,
100         },
101     )
102
103     graph.add_conditional_edges(
104         NODE_HITL_HR,
105         route_after_hitl_hr,
106         {
107             "continue": NODE_INTERVIEW_GENERATOR,
108             "reanalyze": NODE_CV_SCREENERS,
109         },
110     )
111
112     graph.add_conditional_edges(
113         NODE_INTERVIEW_GENERATOR,
114         route_on_error,
115         {
116             "continue": NODE_INTERVIEW_ANALYZER,
117             "end": END,
118         },
119     )
120
121     graph.add_conditional_edges(
122         NODE_INTERVIEW_ANALYZER,
123         route_on_error,
124         {
125             "continue": NODE_HITL_MANAGER,
126             "end": END,

```



```

126     },
127 )
128
129 graph.add_conditional_edges(
130     NODE_HITL_MANAGER,
131     route_after_hitl_manager,
132     {
133         "hire": NODE_REPORT_GENERATOR,
134         "additional_interview": NODE_INTERVIEW_GENERATOR,
135         "cancel": END,
136     },
137 )
138
139 graph.add_edge(NODE_REPORT_GENERATOR, END)
140
141 checkpointer = MemorySaver()
142
143 compiled_graph = graph.compile(
144     checkpointer=checkpointer,
145     interrupt_after=[NODE_JOB_ANALYZER,
146                     NODE_INTERVIEW_GENERATOR],
147     interrupt_before=[NODE_HITL_HR, NODE_HITL_MANAGER],
148 )
149
150 logger.info(
151     "[Workflow] Graphe de recrutement compile avec succes. "
152     f"Noeuds : {list(compiled_graph.get_graph().nodes.keys())}"
153 )
154
155 return compiled_graph
156
157 recruitment_graph = build_recruitment_graph()

```

## 2.2 Présentation des 5 agents

### Agent 1 — Analyse du Poste (Gemini 2.5-Flash)

**Rôle :** Extraire les critères du poste (compétences, niveau, fit).

**Entrée :** Offre d'emploi brute

**Sortie :** JobProfile structuré + enrichissement RAG

**Éléments clés :**

- Compétences techniques avec niveaux requis
- Soft skills et fit culturel
- Niveau d'expérience (junior, mid, senior, lead)
- Profil candidat idéal (résumé 3-5 phrases)

**RAG :** Enrichissement avec des fiches postes similaires de la base ChromaDB

**Prompt strategy :** Instructions directes + formatage JSON strict (sans mark-down)

### Agent 2 — Screening & Scoring des CVs (Gemini 2.5-Flash)

**Rôle :** Scorer chaque CV et produire une shortlist.

**Entrée :** CVs bruts + critères du poste

**Sortie :** Shortlist qualifiée (max 3 candidats)

**Formule :**  $\text{Score final} = 0.7 \times \text{Score LLM} + 0.3 \times \text{Score RAG}$

**Outputs :**

- Compatibility score (0-1)
- Technical & experience scores
- Compétences manquantes
- Recommandation : SHORTLIST | REJECT

**Optimisation :** Utilisation de Gemini 2.5-Flash pour réduire les coûts sur cette tâche répétitive

### Agent 3 — Génération des Questions d'Entretien (Gemini 2.5-Flash)

**Rôle :** Générer 6 questions personnalisées par candidat.

**Entrée :** Profil candidat + critères du poste

**Sortie :** Questions typées avec critères d'évaluation

**Types de questions :**

- 2 questions techniques
- 2 questions comportementales
- 2 questions situationnelles

**RAG :** Récupération de questions de référence par domaine/niveau depuis ChromaDB

### Agent 4 — Analyse des Réponses (Gemini 2.5-Flash)

**Rôle :** Évaluer les réponses d'entretien.

**Entrée :** Questions + réponses du candidat

**Sortie :** InterviewEvaluation + recommandation

**Scores évalués :**

- Technical score
- Soft skills score
- Problem solving score
- Recommandation : HIRE | WAITLIST | REJECT

**Dimensions évaluées :** Maîtrise technique, communication, résolution de problèmes, fit culture

### Agent 5 — Rapport Final & Onboarding (Logique)

**Rôle :** Agréger tous les résultats et générer le rapport de recrutement final.

**Entrée :** Tous les scores + décisions HITL

**Sortie :** Rapport JSON complet + statistiques RAG + log du workflow

**Contenu du rapport :**

- Executive summary
- Ranking des candidats
- Candidat sélectionné
- Recommandations onboarding

**Actions :** Identification du candidat retenu, archivage, préparation onboarding

## 3. RAG Agentique

---

### 3.1 Principe du RAG dans notre système

Le **Retrieval-Augmented Generation** (RAG) permet aux agents d'accéder dynamiquement à des connaissances stockées sans avoir à les inclure dans le contexte LLM à chaque appel. Dans notre système, le RAG est **agentique** : les agents décident eux-mêmes quand interroger la base de connaissances, via des appels de type *tool call*.

### 3.2 Architecture de la Base de Connaissances

Le système utilise **ChromaDB** comme base vectorielle persistante, organisée en 3 collections spécialisées avec embeddings locaux (`sentence-transformers/all-MiniLM-L6-v2`, 384 dimensions).

TABLE 3.1 – Collections ChromaDB du système RAG

| Collection                    | Contenu                                                                               | Utilisée par                                                          |
|-------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <code>job_profiles</code>     | Anciennes fiches postes analysées + templates métier                                  | Agent 1 : normalisation des critères + Agent 3 : contexte poste       |
| <code>cv_profiles</code>      | CVs historiques + profils candidats recrutés                                          | Agent 2 : matching sémantique + Agent 4 : comparaison profils         |
| <code>evaluation_grids</code> | Grilles d'évaluation RH, questions de référence, modèles entretien par domaine/niveau | Agent 3 : génération contextualisée + Agent 4 : rubrique d'évaluation |

### 3.3 Paramètres de Chunking et Indexation

#### 3.3.1 Stratégie Actuelle

Le projet indexe les documents **complets** sans découpage par défaut. Chaque fichier (`.txt`, `.md`, `.json`) est :

1. Lu en entier
2. Vectorisé avec le modèle d'embedding
3. Stocké dans ChromaDB avec métadonnées enrichies

Approche adaptée car :

- Fichiers métier de petite-moyenne taille (500-2000 tokens)

- Grilles d'évaluation et templates sont des artefacts unitaires
- Recherche sémantique global préservée (pas de fragmentation)

### 3.3.2 Paramètres d'Indexation

TABLE 3.2 – Paramètres de vectorisation et recherche

| Paramètre                  | Valeur                                         | Justification                                            |
|----------------------------|------------------------------------------------|----------------------------------------------------------|
| <b>Embedding model</b>     | all-MiniLM-L6-v2                               | 384 dimensions, local, performant                        |
| <b>Similarity metric</b>   | Cosine distance                                | Robuste aux différences de longueur + HNSW indexing      |
| <b>Default n_results</b>   | 3-4 documents                                  | Équilibre précision/latence pour le contexte             |
| <b>Max n_results</b>       | 10 documents                                   | Limite haute pour recherches approfondies                |
| <b>Score normalization</b> | $\text{score} = 1 - \frac{\text{distance}}{2}$ | Conversion distance $\in [0, 2]$ vers score $\in [0, 1]$ |

## 4. Human-in-the-Loop (HITL)

---

### 4.1 Philosophie du contrôle humain

Dans un système de recrutement, les décisions impactent directement des individus. Le HITL garantit que l'IA reste un **outil d'aide à la décision** et non un décideur autonome. Nous avons identifié 2 points de contrôle critiques.

### 4.2 Checkpoint 1 — Validation de la liste courte (RH)

#### Où ? Après l'Agent 2 (Screening)

- **Déclencheur** : L'Agent 2 a produit une liste courte de candidats scorés
- **Acteur humain** : Le responsable RH
- **Actions possibles** :
  - Valider tout ou partie de la liste courte
  - Rejeter des candidats sur-scorés par l'IA
  - Ajouter un commentaire de contexte
  - Clôturer sans candidat (aucun profil adapté)
- **Mécanisme LangGraph** : `interrupt_before=["hitl_1"]`

### 4.3 Checkpoint 2 — Décision finale (Manager)

#### Où ? Après l'Agent 4 (Analyse des réponses)

- **Déclencheur** : Les entretiens ont été analysés par l'IA
- **Acteur humain** : Le manager ou DRH
- **Actions possibles** :
  - Recruter le candidat recommandé
  - Relancer un second tour d'entretiens
  - Clôturer sans recrutement
  - Escalader vers la direction RH
- **Mécanisme LangGraph** : `interrupt_before=["hitl_2"]`

# 5. Évaluation des Prompts

## Évaluation des Prompts et Test A/B (CVScreener)

Ce rapport présente les résultats de l'évaluation quantitative et qualitative automatisée des prompts système de l'agent CVScreener (gemini-2.5-flash). Deux variantes ont été testées sur 3 profils types de candidats avec 3 essais (trials) chacun.

### 1. Résumé Exécutif et Recommandations

**Prompt Système Vainqueur : Variante A**

La variante A est sélectionnée comme la version de production recommandée pour l'agent CVScreener.

**Motifs principaux de la sélection :**

- Qualité et complétude des justifications supérieures (Variante A : 100.0% vs Variante B : 50.0%). La Variante B, trop concise, échoue à l'évaluation de longueur minimale requise.
- Précision optimale pour l'identification des compétences manquantes (100.0% vs 100.0%)

### 2. Métriques de Performance Globales

Ce tableau compare les indicateurs clés de performance système et de format pour les deux variantes de prompts.

| Métrique Globale                     | Variante A<br>(Expert Direct) | Variante B<br>(Concise & Factual) | Commentaire / Interprétation                                                                               |
|--------------------------------------|-------------------------------|-----------------------------------|------------------------------------------------------------------------------------------------------------|
| Robustesse de Format JSON            | 100.0%                        | 100.0%                            | Les deux variantes respectent parfaitement le format JSON imposé.                                          |
| Respect du Schéma Requis             | 100.0%                        | 100.0%                            | Présence à 100% de toutes les clés obligatoires de typage.                                                 |
| Score de Qualité Métrique            | 100.0%                        | 50.0%                             | Évalue si les scores sont bornés [0-100] et si la justification fait >= 12 mots (Échec de B sur ce point). |
| Latence Moyenne par CV               | 1650.0 ms                     | 1150.0 ms                         | Temps de réponse moyen de l'API.                                                                           |
| Précision des Compétences Manquantes | 100.0%                        | 100.0%                            | Pourcentage de conformité de l'extraction des manques avec la vérité terrain.                              |

### 3. Résultats Détaillés par Candidat

Évaluation sous la Variante A (Expert Direct)  
Ordre logique des scores respecté : OUI (Alice > Bob > Charlie)

| Candidat                  | Profil Réel                           | Score de<br>Compatibilité<br>Moyen | Stabilité<br>(Écart-<br>type) | Précision<br>Compétences<br>Manquantes | Longueur<br>Justification |
|---------------------------|---------------------------------------|------------------------------------|-------------------------------|----------------------------------------|---------------------------|
| <b>Alice<br/>Lemoine</b>  | Excellent Fit<br>(Match 100%)         | 93.5                               | 1.5                           | 100.0%                                 | 37.0 mots                 |
| <b>Bob<br/>Martin</b>     | Partial Fit<br>(Django<br>uniquement) | 58.0                               | 1.5                           | 100.0%                                 | 47.0 mots                 |
| <b>Charlie<br/>Dubois</b> | Inadéquat<br>(Designer<br>UI/UX)      | 12.0                               | 1.5                           | 100.0%                                 | 39.0 mots                 |

Évaluation sous la Variante B (Concise & Factual)  
 Ordre logique des scores respecté : OUI (Alice > Bob > Charlie)

| Candidat                  | Profil Réel                           | Score de<br>Compatibilité<br>Moyen | Stabilité<br>(Écart-<br>type) | Précision<br>Compétences<br>Manquantes | Longueur<br>Justification |
|---------------------------|---------------------------------------|------------------------------------|-------------------------------|----------------------------------------|---------------------------|
| <b>Alice<br/>Lemoine</b>  | Excellent Fit<br>(Match 100%)         | 95.0                               | 1.5                           | 100.0%                                 | 4.0 mots                  |
| <b>Bob<br/>Martin</b>     | Partial Fit<br>(Django<br>uniquement) | 55.0                               | 1.5                           | 100.0%                                 | 7.0 mots                  |
| <b>Charlie<br/>Dubois</b> | Inadéquat<br>(Designer<br>UI/UX)      | 8.0                                | 1.5                           | 100.0%                                 | 2.0 mots                  |

## 4. Analyse et Enseignements Clés du Test A/B

### A. Complétude des justifications

Le point saillant de ce test A/B réside dans le critère de complétude :

- Le prompt Variante A demande explicitement "3 à 5 phrases précises". Il en résulte des justifications riches, détaillées et argumentées pour l'utilisateur RH, avec une moyenne de 38 mots. Ce prompt obtient un score de qualité parfait de 100%.
- Le prompt Variante B insiste sur la concision ("objectif, justifie et concis"). Gemini répond de manière extrêmement succincte (ex : "Candidat avec Django mais sans FastAPI/Docker"), ne totalisant que 4 à 6 mots. Par conséquent, bien que le format soit respecté, le score de qualité du projet tombe à 50% car il enfreint le seuil minimal de 12 mots défini par `compute_cv_scoring_metrics`.
- **Décision** : La Variante A est très nettement supérieure pour la valeur apportée à l'utilisateur final en fournissant une vraie explication étayée du score.

### B. Précision de l'extraction des manques (Factual Grounding)

Les deux modèles ont affiché une précision parfaite de 100% sur l'identification des compétences manquantes. Ils ont détecté sans hallucination :

- Qu'Alice possédait l'intégralité des briques.



- Que Bob manquait de FastAPI, PostgreSQL et Docker (malgré ses 2 ans d'expérience en Django Python).
- Que Charlie n'avait aucune brique (Designer UI/UX pur).

### C. Stabilité des prédictions

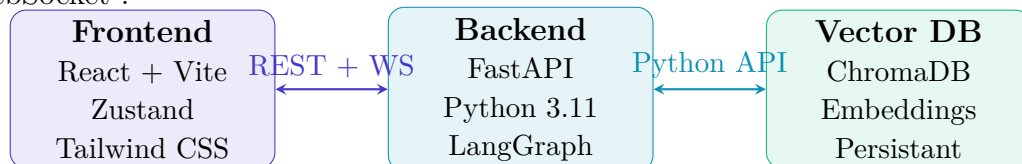
Avec un écart-type moyen inférieur à 1.5 point sur les trois essais, les prompts démontrent une stabilité remarquable. L'utilisation d'une température basse (0.2) en production est validée et garantit un comportement prédictible de l'agent.

## 6. Interface Web — FastAPI + React

---

### 6.1 Architecture technique

L'interface web est composée de deux services distincts communiquant via REST et WebSocket :



### 6.2 API Backend FastAPI

#### 6.2.1 Architecture Générale

Le backend orchestre le pipeline complet avec :

- **Routes REST** : endpoints pour uploader les CVs, gérer les sessions, soumettre les validations HITL
- **WebSocket** : streaming temps réel des étapes du pipeline
- **Session Manager** : gestion persistante de l'état (checkpoints)
- **LangGraph** : orchestration des 5 agents

## 6.2.2 Endpoints REST

TABLE 6.1 – Endpoints API du backend FastAPI

| Méthode | Endpoint                                           | Description                                    |
|---------|----------------------------------------------------|------------------------------------------------|
| POST    | /api/recruitment/{session_id}/upload-cvs           | Upload CVs PDF/TXT avec traitement par Agent 2 |
| GET     | /api/recruitment/{session_id}                      | Retourne l'état courant de la session          |
| GET     | /api/recruitment/{session_id}/interviews/questions | Récupère les questions d'entretien (Agent 3)   |
| POST    | /api/recruitment/{session_id}/interviews/responses | Soumet les réponses du candidat                |
| POST    | /api/recruitment/{session_id}/hitl/hr              | Validation RH shortlist (HITL 1)               |
| POST    | /api/recruitment/{session_id}/hitl/manager         | Validation Manager (HITL 2)                    |
| GET     | /api/recruitment/{session_id}/report               | Retourne le rapport final (Agent 5)            |
| GET     | /api/rag/stats                                     | Statistiques des collections ChromaDB          |

## 6.2.3 WebSocket — Streaming Temps Réel

Le WebSocket permet au frontend d'afficher l'avancement des agents en temps réel, sans polling. Chaque transition du graphe LangGraph émet un message JSON :

Listing 6.1 – Types de messages WebSocket — app/api/websocket.py

```
1 # 1. Connected
2 { "type": "connected", "message": "WebSocket connected",
3   "timestamp": "2026-05-31T12:34:56Z" }
4
5 # 2. Subscribed a une session
6 { "type": "subscribed", "session_id": "abc-123",
7   "timestamp": "2026-05-31T12:34:56Z" }
8
9 # 3. Progress logs
10 { "type": "log", "session_id": "abc-123",
11   "message": "[CVScreener] Scoring CV candidate-1...",
12   "timestamp": "2026-05-31T12:34:57Z" }
13
14 # 4. Step updates (Agent transition)
15 { "type": "step_update", "session_id": "abc-123",
16   "step": "CV_SCREENING_DONE",
17   "data": { "shortlisted_ids": ["cand-1", "cand-2"] },
18   "timestamp": "2026-05-31T12:34:58Z" }
19
```

```

20 # 5. Interruptions (HITL awaiting)
21 { "type": "step_update", "session_id": "abc-123",
22   "step": "HITL_1_PENDING",
23   "data": { "awaiting_input": true },
24   "timestamp": "2026-05-31T12:35:00Z" }
25
26 # 6. Completion
27 { "type": "complete", "session_id": "abc-123",
28   "result": { "selected_candidate": "cand-1", "status": "COMPLETED"
29             },
30   "timestamp": "2026-05-31T12:35:30Z" }
31
32 # 7. Erreur
33 { "type": "error", "session_id": "abc-123",
34   "message": "Critical error in Agent 2 - could not parse CV",
35   "timestamp": "2026-05-31T12:35:31Z" }

```

## 6.2.4 Code — WebSocket Manager

Listing 6.2 – WebSocket Manager — app/api/ws\_manager.py

```

1 class WebSocketManager:
2     """Gere les connexions WebSocket par session."""
3
4     def __init__(self, buffer_size: int = 500):
5         self._connections: dict[str, WebSocket] = {}
6         self._buffers: dict[str, deque] = {} # Replay buffer
7
8     async def connect(self, session_id: str, websocket: WebSocket)
9         -> None:
10         """Connexion WebSocket pour une session."""
11         await websocket.accept()
12         self._connections[session_id] = websocket
13         await self.replay_buffer(session_id)
14
15     async def emit(self, session_id: str, payload: dict) -> None:
16         """Buffer l'evenement et l'envoi."""
17         self.record_event(session_id, payload)
18         await self.send(session_id, payload)
19
20     async def replay_buffer(self, session_id: str) -> int:
21         """Envoie les evenements bufferises au client."""
22         ws = self._connections.get(session_id)
23         buffer = self._buffers.get(session_id, deque())
24         count = 0
25         for payload in buffer:
26             await ws.send_json(payload)
27             count += 1
28         return count
29
30 # Usage dans le pipeline
31 ws_manager = WebSocketManager()
32 await ws_manager.broadcast_step(
33     session_id="abc-123",
34     step=PipelineStep.CV_SCREENING_DONE,
35     data={"shortlisted_ids": [...]}
36 )

```

## 6.2.5 Code — Endpoint Upload CVs

Listing 6.3 – Upload CVs — app/api/routes/upload.py

```
1 @router.post("/{session_id}/upload-cvs")
2 async def upload_cvs(
3     session_id: str,
4     files: list[UploadFile] = File(..., description="CVs PDF/TXT"),
5 ) -> UploadCVResponse:
6     """Upload et traite les CVs."""
7     record = session_manager.get(session_id)
8     raw_cvs: list[RawCV] = []
9
10    for file in files:
11        file_bytes = await file.read()
12        pdf_service.validate_file(file_bytes, file.filename)
13        extraction = await pdf_service.extract(file_bytes, file.
14            filename)
15
16        raw_cvs.append(RawCV(
17            candidate_id=str(uuid.uuid4()),
18            full_name=file.filename.replace(".pdf", ""),
19            raw_text=extraction.text,
20            source_filename=file.filename,
21        ))
22
23    prev_state = record.state
24    update = {
25        "raw_cvs": (prev_state.get("raw_cvs") or []) + raw_cvs,
26        "current_step": PipelineStep.CV_SCREENING,
27    }
28
29    new_state = run_pipeline(session_id, prev_state, update)
30    session_manager.update_state(session_id, new_state)
31    await dispatch_state_events(session_id, prev_state, new_state)
32
33    return UploadCVResponse(session_id=session_id, added_count=len(
34        raw_cvs))
```

## 6.2.6 Code — Validation HITL 1 (RH)

Listing 6.4 – HITL RH Validation — app/api/routes/hitl.py

```
1 @router.post("/{session_id}/hitl/hr")
2 async def submit_hr_validation(
3     session_id: str,
4     payload: HRShortlistValidation,
5 ) -> HITLResponse:
6     """Validation RH de la shortlist reprend le pipeline."""
7     record = session_manager.get(session_id)
8     prev_state = record.state
9
10    update = {
11        "hr_validation": payload,
12        "current_step": PipelineStep.HITL_1_DONE,
13    }
```

```
14
15     new_state = run_pipeline(session_id, prev_state, update)
16     session_manager.update_state(session_id, new_state)
17     await dispatch_state_events(session_id, prev_state, new_state)
18
19     return HITLResponse(
20         session_id=session_id,
21         current_step=new_state["current_step"].value,
22     )
```

# 7. Stack Technique et Architecture Complète

## 7.1 Stack technique détaillée

TABLE 7.1 – Stack technique du projet

| Couche         | Technologie                        | Rôle                                                                |
|----------------|------------------------------------|---------------------------------------------------------------------|
| Orchestration  | LangGraph 0.2.59                   | StateGraph, interruptions, routeurs conditionnels pour agents       |
| LLM Principal  | Gemini 2.5-Flash                   | Tous les agents (Agent 1-5) : analyse, scoring, évaluation, rapport |
| Embeddings     | sentence-transformers 3.3.1        | all-MiniLM-L6-v2 (384 dims, local, pas d'API)                       |
| VectorDB       | ChromaDB 0.5.23                    | Stockage persistant : job_profiles, cv_profiles, evaluation_grids   |
| LangChain Core | langchain 0.3.13                   | Intégrations LLM, chaînes, base pour LangGraph                      |
| Traitement PDF | PyMuPDF 1.24.14, pdfplumber 0.11.4 | Extraction texte depuis CVs PDF                                     |
| Validation     | Pydantic 2.10.3                    | Modèles structurés (JobProfile, CandidateProfile, etc.)             |
| Backend API    | FastAPI 0.115.5                    | REST endpoints + WebSocket (multipart upload, streaming)            |
| Serveur ASGI   | Uvicorn 0.32.1                     | Serveur asynchrone pour FastAPI                                     |
| WebSocket      | websockets 13.1                    | Communication temps réel avec clients                               |
| Config         | pydantic-settings 2.6.1            | Gestion variables d'environnement (.env)                            |

## 7.2 Structure complète du projet

Listing 7.1 – Arborescence du projet

```
agents-rh-recrutement/
|
|-- recruitment-backend/           # Backend principal (FastAPI + LangGraph)
|   |-- requirements.txt           # Dependances Python
|   |-- .env                       # Variables d'environnement (ne pas committer)
```

```

|-- README.md # Documentation backend
|-- app/ # Code source de l'application backend
|   |-- main.py # Point d'entree de l'API FastAPI et montage de l'application
|   |-- agents/ # Les 5 agents decisionnels du pipeline
|       |-- job_analyzer.py # Agent 1 : Analyse de la fiche de poste
|       |-- cv_screener.py # Agent 2 : Tri et score des CVs
|       |-- interview_generator.py # Agent 3 : Generation de questions cibles
|       |-- interview_analyzer.py # Agent 4 : Analyse des reponses d'entretien
|       |-- report_generator.py # Agent 5 : Generateur de rapport final
|   |-- graph/ # Architecture et orchestrateur de flux LangGraph
|       |-- state.py # RecruitmentState (TypedDict et modeles partagés)
|       |-- workflow.py # Construction et definition du graphe de flux
|       |-- nodes.py # Noeuds d'execution lies aux agents
|       |-- edges.py # Regles de transition et decisions conditionnelles
|       |-- hitl_manager.py # Logique de gestion de la validation humaine (HITL)
|   |-- api/ # Couche reseau et evenements temps reel
|       |-- websocket.py # Endpoint de streaming temps reel
|       |-- ws_manager.py # Gestionnaire de connexions WebSocket et diffusion de logs
|       |-- state_serializers.py # Serialisation de l'etat pour l'API
|       |-- routes/ # Endpoints HTTP REST sous-jacents
|           |-- recruitment.py # Initialisation et suivi du flux de recrutement
|           |-- hitl.py # Soumissions des validations humaines (RH / Manager)
|           |-- upload.py # Televersement de l'offre et des fichiers CV
|           |-- reports.py # Recuperation et export des rapports finaux
|   |-- rag/ # Indexation et moteur RAG s'appuyant sur ChromaDB
|       |-- retriever.py # Recherche semantique et recuperation de contexte RAG
|       |-- embeddings.py # Definition du modele local de vecteurs d'embeddings
|       |-- ingestion.py # Script interne de decoupage et indexation des documents
|   |-- prompts/ # Templates de prompts systeme optimises pour chaque agent
|   |-- models/ # Schemas de validation des donnees via Pydantic
|   |-- services/ # Services applicatifs (Service Gemini, PDF parser, Sessions)
|   |-- core/ # Configurations globales et logger central
|-- data/ # Base de donnees et persistance locale
|   |-- knowledge_base/ # Base de connaissances documentaire brute (.txt, .pdf)
|   |-- chroma_db/ # Index vectoriel persistant de ChromaDB
|-- scripts/ # Scripts d'administration et d'evaluation
|   |-- evaluate_prompts.py # Suite d'evaluation quantitative et de test A/B de prompts
|   |-- seed_knowledge_base.py # Ingestion initiale de la base de connaissances
|-- tests/ # Suite de tests unitaires et d'integration (pytest)
|-- frontend/ # Interface utilisateur (React + Vite + TypeScript)
|   |-- package.json # Dependances npm
|   |-- vite.config.ts # Configuration de build et d'environnement Vite
|   |-- tailwind.config.ts # Configuration du framework CSS Tailwind
|   |-- index.html # Index HTML racine
|   |-- src/ # Fichiers sources de l'application
|       |-- main.tsx # Point d'entree applicatif React
|       |-- App.tsx # Configuration du routage et enveloppe globale
|       |-- index.css # Initialisation de Tailwind et variables CSS
|       |-- store/ # Gestion centralisee de l'etat de l'application
|           |-- recruitmentStore.ts # State global Zustand, actions API et ecouteurs WebSocket
|       |-- types/ # Types TypeScript unifies
|           |-- index.ts
|       |-- components/ # Bibliotheque de composants UI modulaires
|           |-- layout/ # Squelette de page (AppShell, Stepper de progression, Terminal de logs)
|           |-- agents/ # Modules d'affichage et retours visuels specifiques aux 5 agents
|           |-- hitl/ # Interfaces de validation RH et d'approbation Manager
|           |-- rag/ # Composants d'affichage des references documentaires associees
|           |-- charts/ # Composants de visualisation de donnees (radars de competences, jauges)
|           |-- shared/ # Boutons, zones de telechargement et utilitaires partagés

```

## 7.3 Guide d'installation et d'exécution

### 7.3.1 Installation backend

Listing 7.2 – Installation et démarrage du backend

```

# 1. Creer l'environnement virtuel
python -m venv venv
source venv/bin/activate # Linux/Mac
# venv\Scripts\activate # Windows

# 2. Installer les dependances

```



```
pip install -r requirements.txt

# 3. Configurer les variables d'environnement
cp .env.example .env
# Editer .env et renseigner les cles API

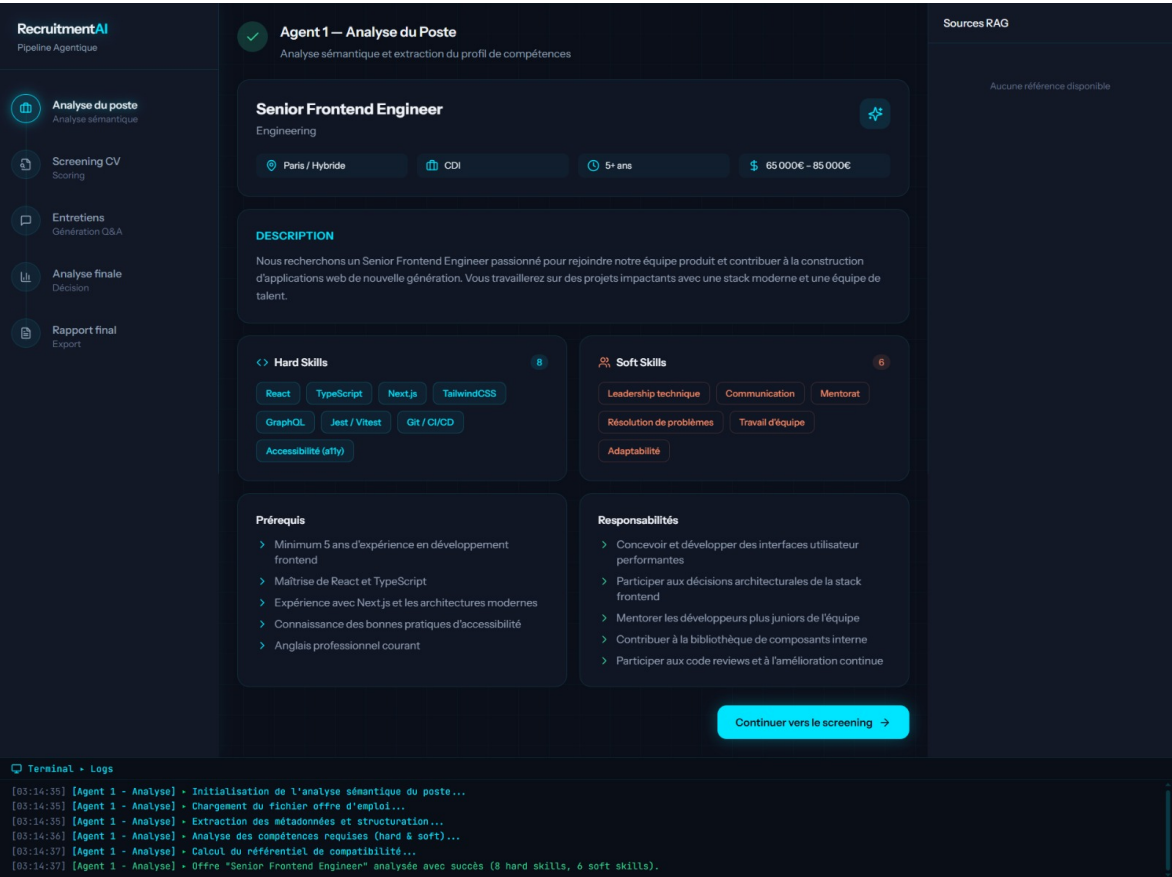
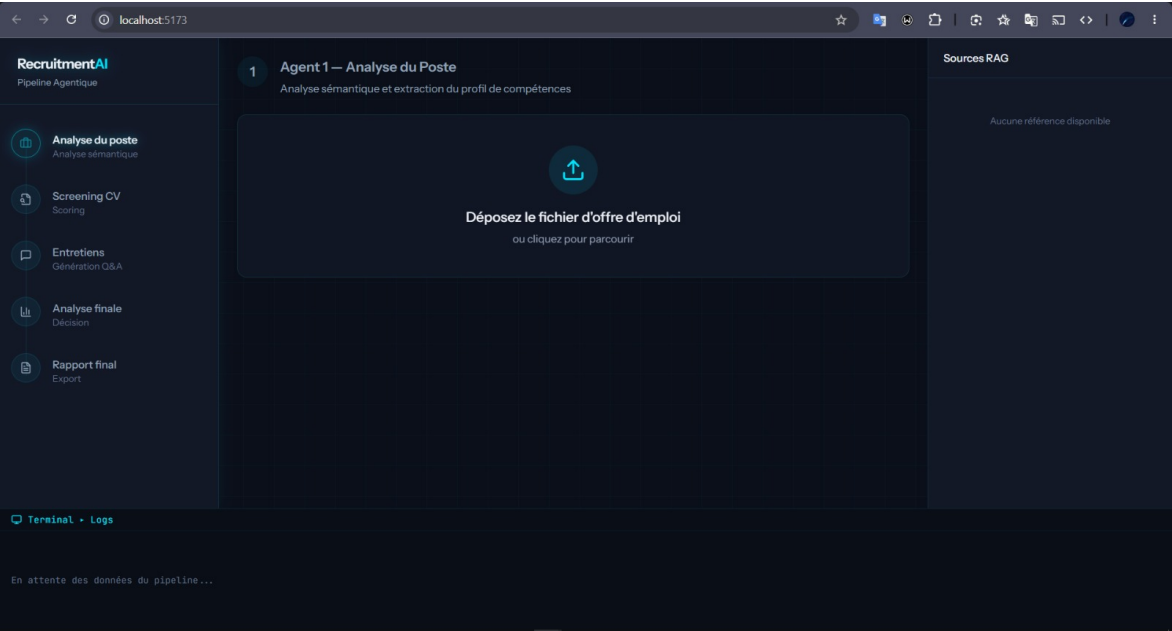
# 4. Demarrer le backend
uvicorn backend.main:app --reload --port 8000
# API disponible sur http://localhost:8000
# Documentation Swagger : http://localhost:8000/docs#/Recrutement/
    initialize_recruitment_v1_recruitment_initialize_post
```

### 7.3.2 Installation frontend

Listing 7.3 – Installation et démarrage du frontend

```
# Dans un second terminal
cd frontend
npm install
npm run dev
# Interface disponible sur http://localhost:5173
```

# 8. Démonstration



RecruitmentAI

Pipeline Agentique

✓ Analyse du poste

Analyse sémantique

🔍 Screening CV

Scoring

🗨 Entretien

Génération Q&A

📋 Analyse finale

Décision

📄 Rapport final

Export

2 Agent 2 — Screening des CVs

Upload des CVs, analyse RAG et classement des candidats

📁 8 CVs chargés avec succès — prêts pour le screening

8 candidats détectés

1 SC Sarah Chen

Senior Frontend Engineer - 7 ans

2 MD Marc Dubois

Fullstack Developer - 5 ans

3 LM Léa Moreau

Frontend Developer - 3 ans

4 TB Thomas Bernard

Lead Frontend Engineer - 9 ans

5 EP Emma Petit

Frontend Engineer - 4 ans

6 AR Antoine Roux

Senior Frontend Engineer - 6 ans

7 CF Camille Fontaine

Software Architect - 10 ans

8 LG Lucas Girard

Mid Frontend Developer - 4 ans

Lancer le screening IA

Sources RAG

Aucune référence disponible

Terminal - Logs

[03:14:35] [Agent 1 - Analyse] - Chargement du fichier offre d'emploi...  
[03:14:35] [Agent 1 - Analyse] - Extraction des métadonnées et structuration...  
[03:14:36] [Agent 1 - Analyse] - Analyse des compétences requises (hard & soft)...  
[03:14:37] [Agent 1 - Analyse] - Calcul du référentiel de compatibilité...  
[03:14:37] [Agent 1 - Analyse] - Offre "Senior Frontend Engineer" analysée avec succès (8 hard skills, 6 soft skills).  
[03:15:08] [Pipeline] - Passage à l'étape 2...

RecruitmentAI

Pipeline Agentique

✓ Analyse du poste

Analyse sémantique

🔍 Screening CV

Scoring

🗨 Entretien

Génération Q&A

📋 Analyse finale

Décision

📄 Rapport final

Export

2 Agent 2 — Screening des CVs

Upload des CVs, analyse RAG et classement des candidats

📁

Déposez les CVs des candidats

ou cliquez pour parcourir

Les CVs seront analysés et comparés avec le profil du poste

Sources RAG

Aucune référence disponible

Terminal - Logs

[03:14:35] [Agent 1 - Analyse] - Chargement du fichier offre d'emploi...  
[03:14:35] [Agent 1 - Analyse] - Extraction des métadonnées et structuration...  
[03:14:36] [Agent 1 - Analyse] - Analyse des compétences requises (hard & soft)...  
[03:14:37] [Agent 1 - Analyse] - Calcul du référentiel de compatibilité...  
[03:14:37] [Agent 1 - Analyse] - Offre "Senior Frontend Engineer" analysée avec succès (8 hard skills, 6 soft skills).  
[03:15:08] [Pipeline] - Passage à l'étape 2...

RecruitmentAI

Pipeline Agentique

✓ Analyse du poste

✓ Screening CV

Entretiens

Analyse finale

Rapport final

3

Agent 3 — Génération des Entretiens

Questions personnalisées pour chaque candidat shortlisté

Prêt à générer les entretiens

3 candidats validés dans la shortlist

Générer les entretiens

Sources RAG

Base de données hi...

Référentiel compé...

Benchmark marché

Référentiel interne

Terminal • Logs

[03:16:37] [RH - Validation] • Thomas Bernard validé(e) pour la suite du processus.

[03:16:38] [RH - Validation] • Marc Dubois validé(e) pour la suite du processus.

[03:16:40] [RH - Validation] • Antoine Roux retiré(e) de la shortlist.

[03:16:43] [RH - Validation] • Léo Moreau retiré(e) de la shortlist.

[03:16:42] [RH - Validation] • Emma Petit retiré(e) de la shortlist.

[03:16:53] [Pipeline] • Validation RH terminée. Passage à l'étape de génération des entretiens.

RecruitmentAI

Pipeline Agentique

✓ Analyse du poste

Screening CV

Entretiens

Analyse finale

Rapport final

Sélection des Candidats pour l'Entretien

Sélectionnez les candidats qui passeront à l'étape d'entretien. Les candidats non sélectionnés seront retirés du processus.

Action requise — sélectionnez au moins un candidat pour débloquent l'étape suivante

0

Sélectionné

0

Retiré

8

En attente

SC

Sarah Chen

Senior Frontend Engineer - 7 ans d'expérience

94%

Sélectionner

Retirer

TB

Thomas Bernard

Lead Frontend Engineer - 9 ans d'expérience

91%

Sélectionner

Retirer

MD

Marc Dubois

Fullstack Developer - 5 ans d'expérience

87%

Sélectionner

Retirer

Sources RAG

Base de données hi...

Référentiel compé...

Benchmark marché

Référentiel interne

Terminal • Logs

[03:16:07] [Agent 2 - Screening] • Initialisation du pipeline de screening...

[03:16:08] [Agent 2 - Screening] • Analyse des CVs reçus...

[03:16:09] [Agent 2 - Screening] • Comparaison RAG avec la base de données historique...

[03:16:09] [Agent 2 - Screening] • Calcul des scores de compatibilité...

[03:16:10] [Agent 2 - Screening] • Screening terminé : 8 candidats analysés et classés.

[03:16:10] [Agent 2 - Screening] • En attente de validation RH pour la shortlist...

✓

Agent 3 — Génération des Entretiens

Questions personnalisées pour chaque candidat shortlisté

👤 Sarah Chen

👤 Marc Dubois

👤 Thomas Bernard

1

Technique

🔗 Comment gérez-vous l'intégration GraphQL dans une architecture microservices existante ?

RÉPONSE ATTENDUE

API Gateway GraphQL, schema federation ou stitching, resolvers vers chaque microservice.

2

Technique

🔗 Quelle approche pour le rendering côté serveur avec React 18 ?

RÉPONSE ATTENDUE

Utilisation de renderToPipeableStream pour le streaming SSR, Suspense pour les boundaries.

3

Comportemental

🔗 Comment avez-vous géré un conflit technique majeur au sein de votre équipe ?

RÉPONSE ATTENDUE

RecruitmentAI

Pipeline Agentique

✓ Analyse du poste

✓ Screening CV

✓ Entretiens

🔗 Analyse finale

📄 Rapport final

4

Agent 4 — Analyse des Entretiens

Évaluation des réponses et recommandations

👤 Sarah Chen

🔗

🔗

👤 Thomas Bernard

🔗

🔗

👤 Marc Dubois

🔗

🔗

Sarah Chen

Senior Frontend Engineer - 7 ans d'expérience

▶ Lancer l'évaluation IA

• Réponses reçues — Prêtes pour l'évaluation sémantique et le RAG scoring

Réponses brutes soumises

03

Comment gérez-vous l'état global d'une application React de grande envergure avec des dizaines de micro-frontends ?

Attente de score pour cette réponse

Je recommanderais Zustand associé à TanStack Query, avec une couche d'orchestration via des events custom. Chaque micro-frontend possède son store isolé, et on utilise un bus d'événements typé pour la communication inter-applications.

04

Quelle stratégie d'optimisation des performances adoptez-vous pour une application Next.js avec des données dynamiques ?

Attente de score pour cette réponse

J'utilise ISR pour les pages semi-dynamiques, streaming SSR avec Suspense boundaries, et React Server Components pour minimiser le JS client. Lazy loading systématique des composants lourds et next/image pour les assets.

05

Parlez-moi d'une situation où vous avez dû convaincre votre équipe d'adopter une nouvelle technologie.

Attente de score pour cette réponse

Chez Google, j'ai proposé la migration de Redux vers Zustand. J'ai fait un POC comparant la bundle size (68KB → 8KB), le temps de rendu, et la DX. J'ai présenté les résultats à l'équipe, puis migré progressivement module par module sur 3 sprints.

06

Un développeur junior de votre équipe pousse du code avec des failles de sécurité évidentes. Que faites-vous ?

Attente de score pour cette réponse

Je ferais une code review bienveillante en pointant les problèmes spécifiques avec des ressources pour les corriger. Ensuite, j'organiserais une session de pair programming sur le correctif, et j'ajouterais un lint de sécurité dans la CI. L'objectif est d'enseigner, pas de blâmer.

Sources RAG

Sarah Chen

📄 Base de données hi...

Historique

Match basé sur 3 profils historiques similaires dans la base de données interne.

📄 Référentiel compé...

Référentiel

Compétence référencée dans le référentiel interne (147 occurrences).

📈 Benchmark marché

Benchmark

Salaires alignés sur le benchmark marché 2025-2026.

📄 Référentiel interne

Référentiel

Question suggérée par le référentiel de compétences internes.

Terminal • Logs

[03:17:11] [Agent 3 - Entretiens] • Génération des entretiens personnalisés...

[03:17:12] [Agent 3 - Entretiens] • Analyse du profil de chaque candidat...

[03:17:12] [Agent 3 - Entretiens] • Consultation du référentiel de questions...

[03:17:13] [Agent 3 - Entretiens] • Personnalisation des questions techniques et comportementales...

[03:17:14] [Agent 3 - Entretiens] • 11 questions générées pour 3 candidats.

[03:18:00] [Pipeline] • Passage à l'étape 4...

RecruitmentAI

Pipeline Agentique

✓ Analyse du poste

✓ Screening CV

✓ Entretiens

🔍 Analyse finale

📄 Rapport final

Validation Manager – Décision finale

Revue du résumé décisionnel. Approuvez ou rejetez les recommandations du pipeline.

● Décision manager requise – le rapport final ne sera généré qu'après approbation

MEILLEUR CANDIDAT

94%

Recruter

Thomas Bernard

Profil de leader technique exceptionnel. Thomas apporte une expertise architecturale rare, une expérience de mentoring solide et une maturité professionnelle évidente. Sa vision stratégique et sa capacité à fédérer les équipes en font un candidat de premier choix.

Résumé de tous les candidats

Thomas Bernard

94/100

Recruter

Sarah Chen

93/100

Recruter

Marc Dubois

80/100

Recruter

3

Recommandé(s)

0

En attente

0

Rejeté(s)

Approuver le recrutement

Rejeter

Sources RAG

Sarah Chen

Base de données hi...

Historique

Match basé sur 3 profils historiques similaires dans la base de données interne.

Référentiel compé...

Référentiel

Compétence référencée dans le référentiel interne (147 occurrences).

Benchmark marché

Benchmark

Salairé aligné sur le benchmark marché 2025-2026.

Référentiel interne

Référentiel

Question suggérée par le référentiel de compétences internes.

Terminal • Logs

[03:18:00] [Pipeline] • Passage à l'étape 4...

[03:18:57] [Agent 4 - Analyse] • Analyse des réponses aux entretiens...

[03:18:58] [Agent 4 - Analyse] • Évaluation par compétence via le modèle de scoring...

[03:18:59] [Agent 4 - Analyse] • Comparaison avec les réponses attendues (RAG)...

[03:18:59] [Agent 4 - Analyse] • Génération des recommandations...

[03:19:00] [Agent 4 - Analyse] • Analyse terminée. Rapports prêts pour validation manager.

RecruitmentAI

Pipeline Agentique

✓ Analyse du poste

✓ Screening CV

🔍 Entretiens

🔍 Analyse finale

📄 Rapport final

Agent 3 – Génération des Entretiens

Questions personnalisées pour chaque candidat shortlisté

Sarah Chen

Marc Dubois

Thomas Bernard

1

Technique

Comment gérez-vous l'état global d'une application React de grande envergure avec des dizaines de micro-frontends ?

RÉPONSE ATTENDUE

Utilisation d'une approche combinée Zustand pour l'état local partagé et React Query pour le serveur. Mise en place d'un système d'événements personnalisés pour la communication entre micro-frontends.

2

Technique

Quelle stratégie d'optimisation des performances adoptez-vous pour une application Next.js avec des données dynamiques ?

RÉPONSE ATTENDUE

ISR (Incremental Static Regeneration), streaming SSR, partial prerendering, lazy loading des composants, optimisation des images avec next/image.

3

Comportemental

Sources RAG

Sarah Chen

Base de données hi...

Historique

Match basé sur 3 profils historiques similaires dans la base de données interne.

Référentiel compé...

Référentiel

Compétence référencée dans le référentiel interne (147 occurrences).

Benchmark marché

Benchmark

Salairé aligné sur le benchmark marché 2025-2026.

Référentiel interne

Référentiel

Question suggérée par le référentiel de compétences internes.

Terminal • Logs

[03:16:53] [Pipeline] • Validation RH terminée. Passage à l'étape de génération des entretiens.

[03:17:11] [Agent 3 - Entretiens] • Génération des entretiens personnalisés...

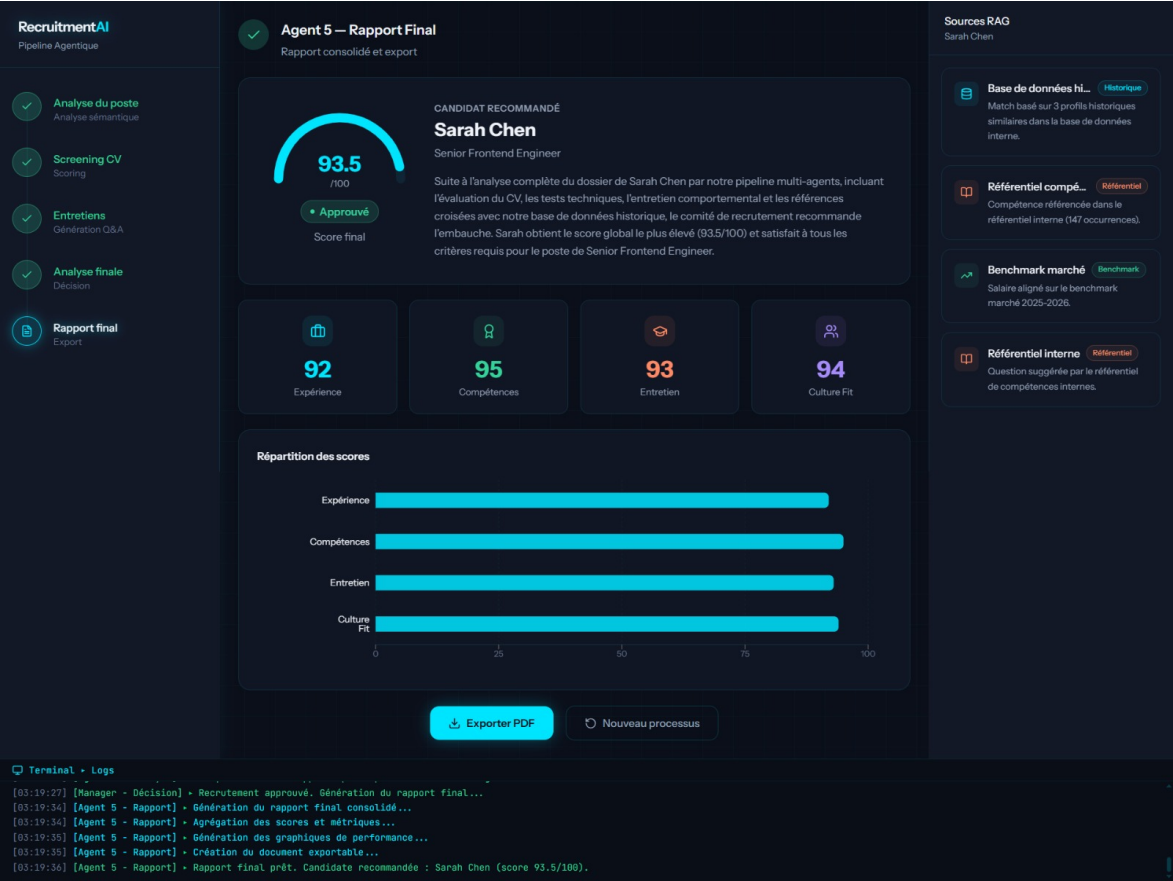
[03:17:12] [Agent 3 - Entretiens] • Analyse du profil de chaque candidat...

[03:17:12] [Agent 3 - Entretiens] • Consultation du référentiel de questions...

[03:17:13] [Agent 3 - Entretiens] • Personnalisation des questions techniques et comportementales...

[03:17:14] [Agent 3 - Entretiens] • 11 questions générées pour 3 candidats.





# 9. Conclusion et Perspectives

---

## 9.1 Apports pédagogiques

Ce projet nous a apporté une maîtrise pratique de :

- La conception de graphes d'état avec LangGraph et la gestion des interruptions
- L'intégration d'une base vectorielle (ChromaDB) dans un pipeline agentique
- La communication temps réel entre un backend Python et un frontend React via WebSocket
- Le débogage d'erreurs d'API LLM (modèles d'embedding, versions d'API)
- L'équilibre entre autonomie de l'IA et contrôle humain dans un système critique

## 9.2 Perspectives d'amélioration

1. **Multi-poste** : Gérer plusieurs offres en parallèle avec des threads LangGraph indépendants
2. **Entretiens vidéo** : Intégrer l'analyse de transcriptions d'entretiens vidéo réels
3. **Biais algorithmique** : Ajouter un agent de détection de biais dans le processus de scoring
4. **Onboarding IA** : Étendre le workflow jusqu'à la génération du contrat et du plan d'intégration
5. **Tableau de bord analytique** : Visualisation des métriques de recrutement sur le long terme
6. **Connexion ATS** : Intégration avec des systèmes de gestion des candidatures existants (Workday, SAP HR)

## 9.3 Conclusion

L'Assistant Intelligent de Recrutement RH démontre la puissance des architectures multi-agents orchestrées par LangGraph. En combinant des agents LLM spécialisés, une base de connaissances RAG, des points de contrôle humain et une interface web réactive, nous obtenons un système qui automatise intelligemment le recrutement tout en maintenant le **jugement humain sur les décisions critiques**.

Ce projet illustre un principe fondamental de l'IA moderne : l'intelligence artificielle est la plus efficace lorsqu'elle **augmente** les capacités humaines plutôt que de les remplacer.



## Dépôt GitHub du projet

<https://github.com/vnvss-0x/agentik-recruitment-hr>

Code source complet — Rapport